SKN35 · LLM 연동 내외부 문서 기반 질의응답 시스템

LangChain 버전 마이그레이션 도우미

문서 근거와 실행 근거로 이중 검증하는 코드 변환 챗봇

[팀 이름] · 발표 [발표자 이름] · [발표일]

문제

ChatGPT가 준 LangChain 코드, 실행하면 깨집니다

```
# LLM이 흔히 주는 코드

from langchain.chat_models import ChatOpenAI
from langchain.chains import LLMChain

# LangChain 1.4.4에서 실행 결과

ImportError: cannot import name 'ChatOpenAI'
ModuleNotFoundError: No module named 'langchain.chains'
```

첫 줄부터 에러

LangChain 1.0에서 패키지가 나뉘고, 옛 체인과 에이전트가 빠졌습니다.

더 큰 문제

LLM은 자기 답이 틀렸다는 것을 **모릅니다.**

원인: 버전 지체 환각

인터넷에는 구버전 코드가 더 많습니다

LLM은 데이터가 많은 쪽을 따라갑니다. 라이브러리는 빠르게 바뀌지만, 학습 데이터 속 코드는 대부분 과거 버전입니다.

42.55%

문서가 부족할 때, LLM 코드 중 목표 버전에서 실행되는 비율

When LLMs Lag Behind, 2026

66.36%

구조화된 문서를 줘도 실행되는 비율

When LLMs Lag Behind, 2026

48~51%

최신 상용 모델의 버전 지정 코드 생성 성공률

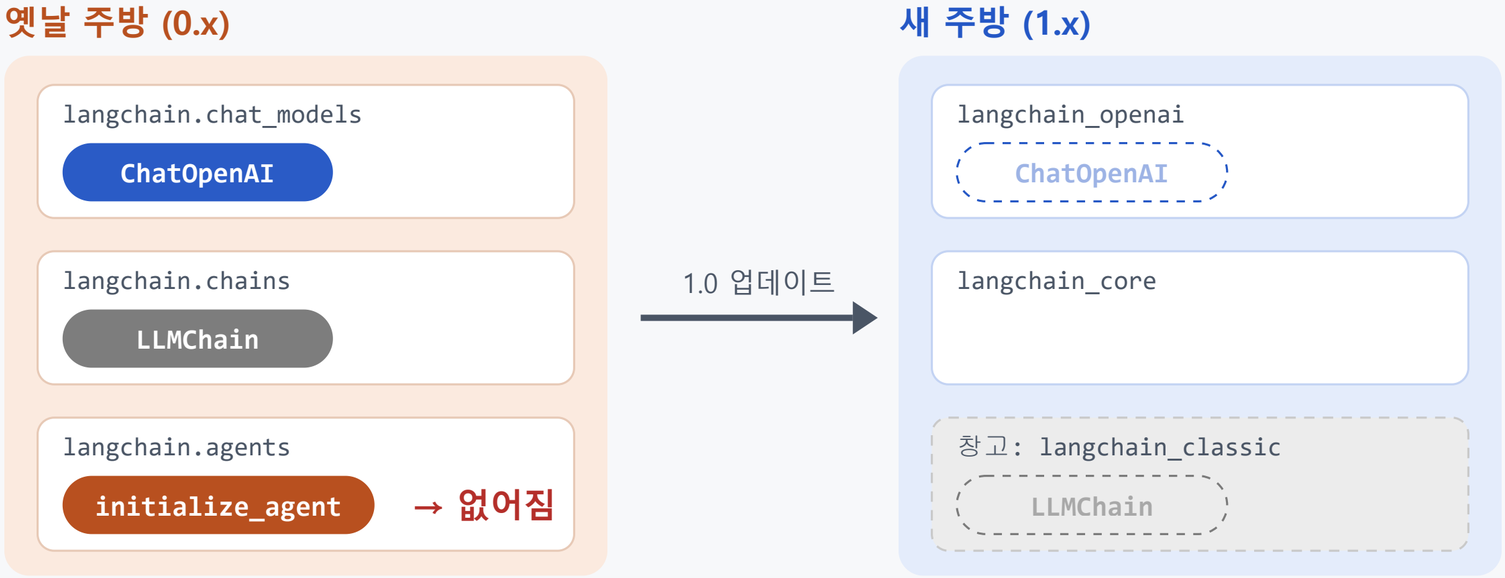
GitChameleon 2.0, 2025

쉽게 보면

도구가 새 서랍으로 이사 갔습니다

LangChain 1.0: 주방을 새로 정리했어요

import = "어느 서랍에서 어떤 도구를 꺼내라" · 도구가 다른 서랍으로 이사 갔습니다



옛날 서랍을 열면 → ImportError "그런 도구 없어요" · ModuleNotFoundError "그런 서랍 없어요"

해결

문서로 한 번, 실행으로 한 번 더 확인합니다

같은 옛날 레시피, 두 요리사

일반 AI

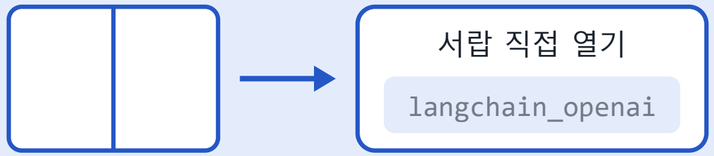


기억 속 옛날 주방만 보고
레시피를 고침

```
from langchain.chat_models ...
```

✗ 실행하면 에러
그런데 틀린 줄 모르고 그대로 끝

우리 프로그램



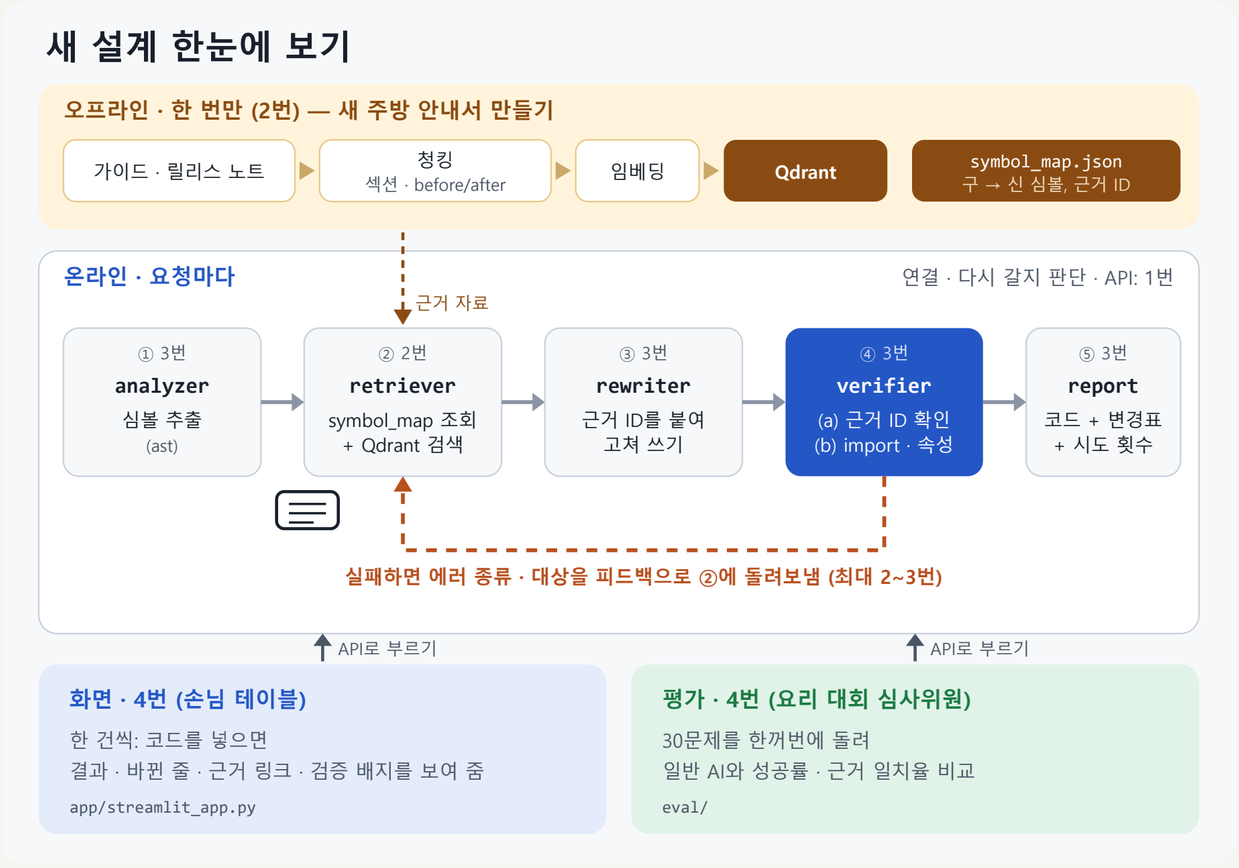
① 안내서 보기
문서 근거

② 실제로 확인
실행 근거

도구 있음 → 통과
없으면 에러를 힌트로
다시 찾아서 고침 (최대 2~3번)

시스템 구조

안내서를 미리 만들고, 다섯 단계로 고칩니다



한 번만: 안내서

가이드 · 릴리스 노트로 Qdrant와 symbol_map(구 → 신 심볼)

핵심: verifier

(a) 근거 ID가 실제로 있는지 (b) import · 속성 검증

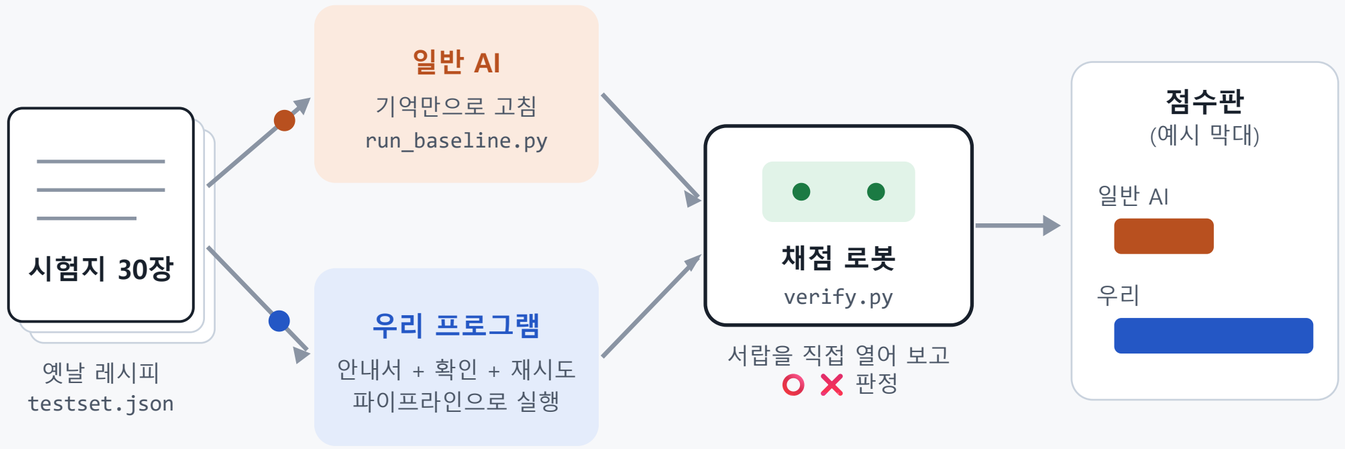
실패하면 되돌아감

에러 종류 · 대상을 피드백으로 다시 검색 (최대 2~3번)

실험 설계

같은 코드 30개, 같은 조건으로 비교했습니다

내 임무: 요리 대회 심사위원 (4번)



공정한 시합 규칙

- 같은 AI 두뇌(모델)
- 같은 문제 30개
- 같은 채점 로봇
- 꼼수는 실격

쉬움 10개

import 경로만 바꿈

중간 10개

체인 구조 변경

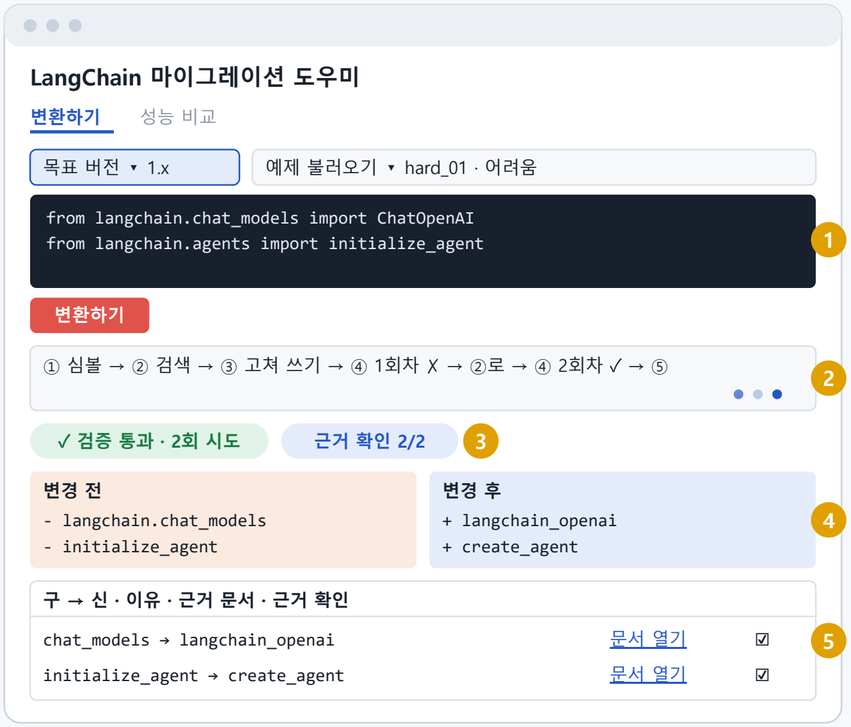
어려움 10개

에이전트·메모리 변경

데모

결과와 근거를 한 화면에서 확인합니다

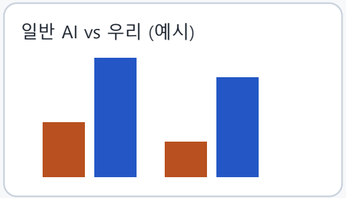
화면 설계도 (4번)



변환하기 탭

① 목표 버전 고르고 코드 넣기
② 파이프라인 진행 기록
(되돌아간 과정까지 보임)
③ 검증 배지 · 근거 확인 배지
④ 변경 전 · 후와 바뀐 줄
⑤ 변경표와 근거 문서 링크

성능 비교 탭



eval/charts 의 그래프 3장과
요약표를 그대로 보여 줌

`app/streamlit_app.py`

1 · 목표 버전 + 구버전 코드

LangChain 1.x · initialize_agent 에이전트 [hard_01]

2 · 1회차 실패 → 재검색

진행 기록에 되돌아간 과정이 보임

3 · 통과, 근거와 함께

검증 배지, 바뀐 줄, 근거 문서 링크

결과

어려운 코드일수록 차이가 커졌습니다

우리 시스템

[__]%

일반 LLM (baseline)

[__]%

전체 성공률 (import · 속성 검증), 테스트 코드 30개 기준

근거 일치율 [__]%

[그래프 삽입]
charts/1_success_by_difficulty.png

검증 루프 효과

실행 검증이 성공률을 [+__%p] 끌어올렸습니다

[그래프 삽입]
charts/2_retry_effect.png

1회차 성공률

[__]%

최종 성공률 (최대 [2~3]회)

[__]%

성공까지 평균 시도

[__]회

실험 중 발견

통과했다고 끝이 아니었습니다

지원 종료 중인 패키지

langchain-community는 import만 해도 "지원 종료 중" 경고를 냅니다. 지금은 통과해도 곧 깨질 수 있습니다.

2026-10-12 확인 · langchain-community 0.4.2

classic 우회

LLMChain은 물론 initialize_agent 같은 에이전트도 langchain_classic에서 가져오면 import는 통과합니다. 하지만 최신 방식은 아닙니다.

[그래프 삽입]

charts/3_success_quality.png

한계와 다음 단계

통과가 곧 정답은 아닙니다

한계	다음 단계
import와 속성 이름까지 확인하지만, 인자나 실행 결과는 확인 못함	가짜 LLM으로 실제 실행까지 테스트
테스트 30개, 팀이 직접 구축	공개 저장소의 실사용 코드로 확장
구버전 환경 동작 확인은 생략	구버전 환경을 따로 만들어 교차 확인
LangChain만 대상	pydantic, OpenAI SDK 등으로 확장

가치

라이브러리가 바뀔 때마다, 근거와 함께 확인된 코드

시간 절감

개발자가 에러를 직접 추적하는 시간을 줄임
[__분 → __분]

신뢰

근거 문서 링크와 검증 통과 여부를 함께 제시

확장

문서와 검증 환경만 바꾸면 다른 라이브러리에도 적용

팀

1번
그래프 · 백엔드
State · 연결 · 재시도

2번
데이터 · 검색
문서 · Qdrant · symbol_map

3번
변환 · 검증
변환 · 검증 로직

4번
화면 · 평가 · 발표
화면 · 실험 · 발표

감사합니다

질문 받겠습니다 · github.com/SKN35shimsungwook/langchain-migration-helper